VPTQ: Extreme Low-bit Vector Post-Training Quantization for Large Language Models

Yifei Liu, Jicheng Wen, Yang Wang, Shengyu Ye et al.
USTC & Microsoft Research, 2024

Presented by Doyoon Kim

Spring 2026

Outline

1. **Motivation** — LLMs are too large
2. **Background** — PTQ and Vector Quantization
3. **VPTQ Methods** — Four key components
4. **End-to-End Algorithm**
5. **Experimental Results**

LLMs Are Too Large to Deploy

- LLM size grows rapidly with capability
  - LLaMA-2 70B in FP16 $\approx$ **140 GB** RAM
  - Requires multiple high-end GPUs $\rightarrow$ high deployment cost
- **Weight-only Quantization**: compress weights to low-bit without retraining
  - Reduces memory and improves inference throughput
  - FP16 (16-bit) $\rightarrow$ 2-bit: memory reduced by $8\times$
- Key question: **Can we push to 2-bit while keeping usable accuracy?**

The Limit of Scalar Quantization

- **Scalar quantization**: round each weight independently to a low-bit value
- 2-bit $\rightarrow$ only **4 distinct values**: $\{0, 1, 2, 3\}$
- GPTQ and similar methods work well at 3–4 bit, but **accuracy collapses at 2-bit**

Example: 2-bit scalar quantization

Original	Quantized	Error
0.73	1	0.27
0.31	0	0.31
-0.68	-1	0.32
0.95	1	0.05

Large errors; cannot capture the diversity of weight values

The Intuition Behind Vector Quantization

- **Key idea**: group multiple weights together and represent them as a single entry in a *codebook*

Example: vector length $v = 2$, codebook size $k = 4$

Index	Centroid
0	$[-0.80,\ -0.25]$
1	$[-0.10,\ \ 0.60]$
2	$[\ \ 0.30,\ -0.50]$
3	$[\ \ 0.72,\ \ 0.28]$

$[0.70,\ 0.30] \xrightarrow{\text{encode}}$ **index 3**

index $3 \xrightarrow{\text{decode}} [0.72,\ 0.28]$

Scalar vs. VQ at the same 2-bit budget

- Scalar 2-bit: each weight has **4** possible values
- VQ ($v = 2$, $k = 16$): each 2D vector is chosen from **16** learned 2D centroids
- Both use **2 bits/weight**, but VQ represents weights **jointly** rather than independently
- $\rightarrow$ exploits correlations across weights for richer representation

Post-Training Quantization: Deriving the Objective

Goal: quantize $\mathbf{W} \to \hat{\mathbf{W}}$ with minimal change to the task loss $\mathcal{L}$

Taylor-expand the loss change around the original weights:

$$\mathcal{L}(\hat{\mathbf{W}}) - \mathcal{L}(\mathbf{W}) \approx \underbrace{g(\mathbf{W})^T}_{\text{gradient}} \Delta\mathbf{W} + \frac{1}{2} \Delta\mathbf{W}^T \underbrace{H(\mathbf{W})}_{\text{Hessian}} \Delta\mathbf{W} \quad (\Delta\mathbf{W} = \hat{\mathbf{W}} - \mathbf{W})$$

Assumption: the pre-trained model is near a local minimum

$$g(\mathbf{W}) \approx \mathbf{0} \quad \Rightarrow \quad \text{1st-order term drops out}$$

PTQ objective:

$$\boxed{\arg\min_{\Delta\mathbf{W}} \; \Delta\mathbf{W}^T H(\mathbf{W}) \, \Delta\mathbf{W}}$$

- H_{ii} large $\to$ weight i strongly affects loss $\to$ quantize it **precisely**
- H_{ii} small $\to$ weight i is insensitive $\to$ can tolerate larger error

Computing the Hessian in Practice

Challenge: computing the full Hessian of the task loss is intractable for LLMs

Layer-wise approximation (GPTQ):

- Quantize one layer at a time; treat each layer's output error as the local loss

$$\mathcal{L}_\ell \;=\; \|\mathbf{W}\mathbf{X} - \hat{\mathbf{W}}\mathbf{X}\|_F^2$$

- $\mathbf{X}$: layer input collected from calibration data
- This is a quadratic in $\Delta\mathbf{W}$, so its Hessian is exact and cheap:

$$H \;=\; 2\,\mathbf{X}\mathbf{X}^T$$

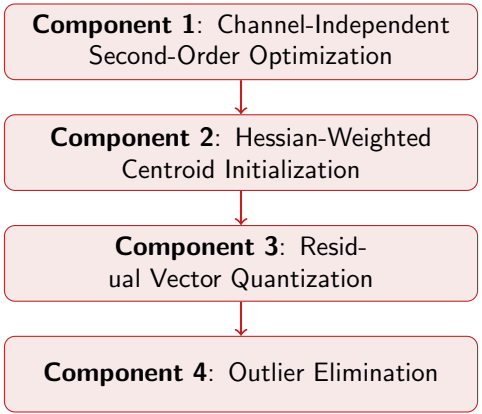
Limitations of Existing VQ Methods

	VPTQ	AQLM	QuIP#	GPTVQ	GPTQ
Effective Bitwidth	⇑	↑	↑	↑	↓
Accuracy @ Low-bit	⇑	↑	↑	↓	↓
Quantization Speed	⇑	⇓	—	↑	⇑
Inference Throughput	⇑	↑	↓	—	↑

⇑: best, ↑: good, ↓: poor

- **GPTVQ**: quantizes v columns simultaneously $\rightarrow$ errors accumulate $\rightarrow$ short vectors only
- **AQLM**: requires gradient backprop $\rightarrow$ GPU-hours (>11h for 7B)
- **QuIP#**: Hadamard transform at inference $\rightarrow$ $O(n^2)$ overhead per operator
- **VPTQ**: addresses all of the above simultaneously

VPTQ: Overview of Four Components



Component 1: Channel-Independent Second-Order Optimization

- **Problem with GPTVQ**: quantizes v columns jointly
  - Quantization errors inside the current block cannot be corrected immediately
  - Errors accumulate as vector length grows
  - This limits GPTVQ to relatively short vectors
- **VPTQ idea**: quantize *one column at a time*
  - Quantize column $A \rightarrow$ immediately propagate its error to the remaining columns
  - Then quantize B, then C, then D, ...
  - This reduces error accumulation and supports longer vectors / larger codebooks
- **Key assumption**: the Hessian is approximately diagonally dominant

$$\Rightarrow \text{inter-column interactions are weaker than same-column effects}$$

Component 1: Why Nearest-Centroid Search Appears

- Start from the Hessian-aware PTQ objective:

$$\arg\min_{\Delta \mathbf{W}} \ \Delta \mathbf{W}^T H \, \Delta \mathbf{W}$$

- For a fixed column q, VPTQ applies the Lagrange method and obtains

$$\Delta L_q = \frac{\|\hat{\mathbf{W}}_{:,q} - \mathbf{W}_{:,q}\|_2^2}{2H_{qq}^{-1}}$$

- Thus, for column q, the loss increase is proportional to the reconstruction error of that column
- In VQ, column q is split into subvectors $\mathbf{v}_j$, each represented by a centroid $\mathcal{C}_{i_j}$

$$\|\hat{\mathbf{W}}_{:,q} - \mathbf{W}_{:,q}\|_2^2 = \sum_j \|\mathbf{v}_j - \mathcal{C}_{i_j}\|_2^2$$

Component 1: Why Nearest-Centroid Search Appears

- Substituting the VQ form into the second-order loss gives

$$\Delta L_q = \frac{\sum_j \|\mathbf{v}_j - \mathcal{C}_{i_j}\|_2^2}{2H_{qq}^{-1}}$$

- For a fixed column q, H_{qq}^{-1} is a **constant**
- Therefore it does not change the minimizer
- The optimization reduces to minimizing Euclidean reconstruction error:

$$\arg\min_i \|\mathbf{v} - \mathcal{C}_i\|^2$$

- This is exactly the **nearest-centroid assignment** in K-means / VQ
- **Interpretation**: once the second-order objective is fixed, choosing the centroid becomes a simple Euclidean lookup

Component 1: Error Propagation Still Requires the Hessian

- The Hessian does **not** disappear from the algorithm
- It is used in two places:
  1. **Defining the second-order PTQ objective**

$$\Delta \mathbf{W}^T H \, \Delta \mathbf{W}$$

  2. **Propagating quantization error to the remaining columns**

$$E_{:,n} = \frac{\mathbf{W}_{:,n} - \hat{\mathbf{W}}_{:,n}}{H^{-1}_{nn}}$$

$$\mathbf{W}_{:,\, n:s+B} \leftarrow \mathbf{W}_{:,\, n:s+B} - E_{:,n} H^{-1}_{n,\, n:s+B}$$

- So:
  - **Centroid selection** becomes Euclidean nearest-neighbor
  - **Error correction / propagation** remains Hessian-aware

Component 2: Hessian-Weighted Centroid Initialization

- K-means needs good initial centroids. VPTQ uses a **Weighted K-means** objective:

$$\min_{\{\mathcal{C}_j\}} \sum_i h_{ii} \left\| \mathbf{w}_i - \mathcal{C}_{\sigma(i)} \right\|^2$$

- $h_{ii} = H_{ii}$: Hessian diagonal — how much weight i impacts the loss
- $\sigma(i)$: centroid assignment for weight i

Weighted centroid update:

$$\mathcal{C}_j \leftarrow \frac{\sum_{i:\,\sigma(i)=j} h_{ii}\,\mathbf{w}_i}{\sum_{i:\,\sigma(i)=j} h_{ii}}$$

Component 2: Hessian-Weighted Centroid Initialization

Example: 4 weights, 2 centroids

weight	value	h_{ii}
w_1	0.04	10.0
w_4	1.00	0.2

Standard K-means centroid:
$\frac{0.04+1.00}{2} = 0.52$

Weighted centroid A:
$\frac{10.0\times0.04 + 0.2\times1.00}{10.0+0.2} = \frac{0.4+0.2}{10.2} \approx 0.059$

High-h_{ii} weights pull the centroid more strongly toward themselves

Component 3: Residual Vector Quantization (RVQ)

- **Idea**: quantize the *residual error* of the first VQ stage again with a second codebook

Example: vector $\mathbf{v} = [0.70,\ 0.30]$

Stage	Value	Stored
Original	$[0.70,\ 0.30]$	—
1st VQ result $Q(\mathbf{v})$	$[0.72,\ 0.28]$	index i_1
Residual $\mathbf{v}_{\text{res}} = \mathbf{v} - Q(\mathbf{v})$	$[-0.02,\ 0.02]$	—
2nd VQ result $Q(\mathbf{v}_{\text{res}})$	$[-0.019,\ 0.021]$	index i_2
Final reconstruction	$[0.701,\ 0.301]$	$i_1 + i_2$ only

- Additional cost: only the size of i_2 (small second codebook)
- Large accuracy gain for small bit overhead
- GPTVQ does *not* support RVQ — a key advantage of VPTQ

Component 4: Outlier Elimination

- **Problem**: $\sim$1% of weights are very large (outliers), dominating quantization error

Example: weight matrix with outliers

Without outlier handling

Weight range	Fraction
$[-0.5,\ 0.5]$ (normal)	99%
± 10.0 (outlier)	1%

Codebook must span $[-10,\ 10]$
$\rightarrow$ centroids spread too wide
$\rightarrow$ normal weights poorly represented

VPTQ: separate outlier codebook

Outlier codebook: top 1% weights
$\rightarrow$ dedicated centroids for $[-10,\ 10]$

Main codebook: remaining 99%
$\rightarrow$ dedicated centroids for $[-0.5,\ 0.5]$

$\rightarrow$ both groups represented precisely

- Outliers identified by large Hessian diagonal values
- Outlier fraction $N\%$ is tunable (typically 1%)

End-to-End Quantization Algorithm

Per-layer procedure:

1. **Outlier handling**
   Separate top $N\%$ outlier columns
   $\rightarrow$ quantize with outlier codebook
2. **Main VPTQ**
   Hessian-weighted centroid init
   $\rightarrow$ build codebook via K-means
   $\rightarrow$ column-wise independent quantization
3. **Residual VQ** (optional)
   Compress residual error with a second codebook
4. **Layer fine-tuning** (optional)
   Lightweight re-optimization of current layer

At inference:

- Store indices + codebook instead of FP16 weights
- Dequantization = **codebook lookup only**
- No Hadamard transform required
- $\rightarrow$ **1.6–1.8**$\times$ faster inference vs. SOTA

Parallelism:

- Layers are independent $\rightarrow$ full GPU parallelism
- Quantization is **5–10**$\times$ faster than AQLM

Experimental Setup

Models

- LLaMA-2: 7B, 13B, 70B
- LLaMA-3: 8B, 70B
- Mistral-7B

Metrics

- WikiText-2 perplexity ↓
- C4 perplexity ↓
- 5 zero-shot QA tasks ↑
  (PIQA, HellaSwag, WinoGrande, ARC-easy, ARC-challenge)

Baselines

- GPTQ
- GPTVQ
- QuIP#
- AQLM
- DB-LLM

Calibration data

- 128 segments from C4 dataset

LLaMA-2 7B: 2-bit Results

Method	bit	W2↓	C4↓	AvgQA↑	tok/s↑	mem(GB)↓	cost(h)↓
FP16	16	5.12	6.63	62.2	38.32	27.22	N/A
GPTQ	2.125	50.75	36.76	39.16	19.59	4.42	0.2
GPTVQ	2.25	6.71	9.9	56.14	N/A	N/A	1.5
DB-LLM	2.01	7.23	9.62	55.1	N/A	N/A	N/A
QuIP#	2	6.19	8.16	**58.2**	4.4	2.25	N/A
AQLM	2.02	6.64	8.56	56.5	19.4	**2.16**	N/A
AQLM	2.29	6.29	8.11	58.6	19.6	2.4	11.07
VPTQ	2.02	**6.13**	**8.07**	**58.2**	**39.9**	2.28	**2**
VPTQ	2.26	**5.95**	**7.87**	**59.4**	35.7	2.48	2.2

- **Perplexity**: VPTQ 6.13 vs. AQLM 6.64 (lower is better)
- **Throughput**: VPTQ 39.9 tok/s vs. AQLM 19.4 ($\approx 2\times$), QuIP# 4.4 ($\approx 9\times$)
- **Quantization time**: AQLM 11h vs. VPTQ **2h**

LLaMA-3 and Mistral: 2-bit Results

Method	bit	W2↓	AvgQA↑
LLaMA-3 8B			
FP16	16	6.14	68.6
QuIP	2	85.1	36.8
DB-LLM	2	13.6	51.7
GPTQ	2	2.10×10^{2}	36.2
VPTQ	2.08	**9.29**	**60.2**
LLaMA-3 70B			
FP16	16	2.9	75.3
GPTQ	2	11.9	45.4
VPTQ	2.02	**5.6**	**70.9**

Method	bit	W2↓	C4↓	AvgQA↑
Mistral-7B				
FP16	16.0	4.77	5.71	68.6
QuIP#	2.01	6.02	6.84	62.2
AQLM	2.01	6.32	6.93	62.2
GPTQ	2.125	1535	164	44.5
GPTVQ	2.25	8.99	18.6	57.7
VPTQ	2.04	**5.64**	**6.43**	**63.2**

- LLaMA-3: competing methods **collapse** at 2-bit; VPTQ remains stable
- Mistral: VPTQ outperforms QuIP# and AQLM across all metrics

Conclusion

Four components of VPTQ

1. **Channel-Independent Opt.**
   Column-wise quantization prevents error accumulation
2. **Hessian-Weighted Init.**
   Centroid placement guided by loss sensitivity
3. **Residual VQ**
   Re-compresses residual for better accuracy
4. **Outlier Elimination**
   Separates outliers to protect main codebook

Key results

- Perplexity reduced by **0.01–7.34** over SOTA at 2-bit
- QA accuracy improved by **0.8–22%**
- **1.6–1.8**× faster inference
- Uses only **10–18%** of quantization time vs. AQLM

Limitations

- Fine-tuning 70B models requires more GPU resources

VPTQ makes practical 2-bit LLM deployment a reality